

REPORT

Design and Analysis of Algorithms (DAA)

Mock Interview / Viva Voce Evaluation

1. Student Details

Particular	Details
Name	Shaik Reehan
Register Number	192565071
Course	Design and Analysis of Algorithms
Evaluation Type	Mock Interview / Viva Voce
Academic Activity	CO5 – AT3 DAA Mock Interview
Date	31 August 2026

2. Introduction

The final evaluation was conducted in the form of a **DAA mock interview** focusing on fundamental concepts of algorithm design, complexity analysis, backtracking, graph algorithms, dynamic programming, greedy algorithms, and computational complexity.

The purpose of the evaluation was to assess the ability to:

- Explain algorithmic concepts clearly.
- Apply appropriate algorithms to given problems.
- Analyze time and space complexity.
- Understand recursive and backtracking techniques.
- Explain P, NP, NP-Hard, and NP-Complete problems.
- Apply the Master Theorem to recurrence relations.
- Compare different algorithmic approaches.
- Explain graph-based problems and their constraints.

The evaluation consisted of **10 technical questions**.

3. Objectives

The main objectives of the final evaluation were:

1. To test fundamental knowledge of Design and Analysis of Algorithms.
 2. To evaluate understanding of algorithmic problem-solving techniques.
 3. To assess knowledge of backtracking and pruning.
 4. To analyze recurrence relations and algorithm complexity.
 5. To understand computational complexity classes.
 6. To evaluate understanding of graph algorithms.
 7. To compare Greedy and Dynamic Programming approaches.
 8. To assess the ability to explain solutions during Viva Voce.
-

4. Topics Covered

The following topics were evaluated:

- Backtracking
 - State-Space Trees
 - N-Queens Problem
 - Subset Sum
 - Graph Coloring
 - P, NP, NP-Hard and NP-Complete
 - Polynomial-Time Reduction
 - Master Theorem
 - Greedy vs Dynamic Programming
 - Hamiltonian Cycle
-

5. Question-Wise Evaluation

Question 1 – Backtracking

Question

What is backtracking? Explain how it differs from brute-force search and give an example.

Response Summary

Backtracking was explained as a recursive technique that builds a solution step-by-step and abandons a partial solution as soon as it is determined that the solution cannot become valid.

The difference from brute force was explained using **pruning**.

Examples given:

- Sudoku
- Maze solving

Evaluation

The response correctly identified:

- Recursive solution construction.
- Difference between brute force and backtracking.
- Pruning of invalid branches.
- Practical applications of backtracking.

Performance

Excellent

6. Question 2 – State-Space Tree

Question

What is a state-space tree and how does pruning work?

Response Summary

A state-space tree was described as a representation of possible partial and complete solutions.

The response correctly explained:

- **Node:** Represents a state or partial solution.
- **Branch:** Represents a choice/action.
- **Pruning:** Removes a subtree when the current state violates a constraint.

Evaluation

The explanation demonstrated a clear understanding of the relationship between recursive choices and the search space.

Performance

Excellent

7. Question 3 – N-Queens

Question

How can we determine whether placing a queen is safe?

Response Summary

Three constraints were identified:

1. Same column.
2. Upper-left diagonal.
3. Upper-right diagonal.

The response also correctly explained that when queens are placed row-by-row, previously occupied rows need to be checked.

Evaluation

The constraints were correctly identified and explained.

Performance

Excellent

8. Question 4 – Subset Sum

Question

For {3, 5, 7, 10} with target 15, explain the backtracking approach.

Response Summary

The response explained the **include/exclude decision** for each element.

A valid solution was identified:

$$3 + 5 + 7 = 15$$

Therefore:

$\{3, 5, 7\}$

is a valid subset.

The following pruning conditions were explained:

- Sum exceeds target.
- Target is reached.
- No elements remain.

Evaluation

The answer demonstrated good understanding of recursive branching and pruning.

Performance

Excellent

9. Question 5 – Graph Coloring

Question

How can backtracking be used to solve graph coloring?

Response Summary

The response explained that before assigning a color to a vertex, the algorithm checks its adjacent vertices.

If all available colors create conflicts, the algorithm backtracks to the previous vertex and tries another color.

Evaluation

The response correctly explained:

- Constraint checking.
- Color assignment.
- Conflict detection.
- Backtracking when no valid color remains.

Performance

Very Good

10. Question 6 – P, NP, NP-Hard and NP-Complete

Question

Explain the difference between P, NP, NP-Hard and NP-Complete.

Response Summary

The response explained:

P

Problems that can be solved efficiently in polynomial time.

NP

Problems for which a proposed solution can be verified efficiently.

NP-Hard

Problems that are at least as hard as the hardest problems in NP.

NP-Complete

Problems that are both:

$$NP \cap NP\text{-Hard}$$

The response also correctly stated:

$$P \subseteq NP$$

Evaluation

The fundamental relationships were correctly understood.

Performance

Very Good

11. Question 7 – Polynomial-Time Reduction

Question

What is polynomial-time reduction and why is it important?

Response Summary

Polynomial-time reduction was explained as transforming an instance of one problem into another problem efficiently.

The key idea was correctly identified:

If problem A can be reduced to problem B efficiently, an efficient solution for B can be used to solve A.

The response also explained its importance in proving NP-Completeness.

Evaluation

The answer demonstrated good understanding of reduction and its role in complexity theory.

Performance

Very Good

12. Question 8 – Master Theorem

Question

Solve:

$$T(n) = 2T(n/2) + n$$

using the Master Theorem.

Response

The standard form is:

$$T(n) = aT(n/b) + f(n)$$

The values were correctly identified:

$$\begin{aligned}a &= 2 \\ b &= 2 \\ f(n) &= n\end{aligned}$$

Then:

$$\log_b a = \log_2 2 = 1$$

Therefore:

$$n^{\log_b a} = n$$

Since:

$$f(n) = \Theta(n^{\log_b a})$$

Master Theorem Case 2 applies.

Hence:

$$\boxed{T(n) = \Theta(n \log n)}$$

Evaluation

The solution was completely correct.

Performance

Excellent

13. Question 9 – Greedy vs Dynamic Programming

Question

What is the difference between Greedy and Dynamic Programming?

Response Summary

The response explained that:

- Greedy selects the locally best option.
- Dynamic Programming solves overlapping subproblems and stores their results.

The example of **Fractional Knapsack** was correctly selected.

The reason Greedy works was also correctly explained using the **value-to-weight ratio**.

Evaluation

The conceptual difference and example were correctly explained.

Performance

Excellent

14. Question 10 – Hamiltonian Cycle

Question

What is a Hamiltonian Cycle and how can backtracking be used to find it?

Response Summary

A Hamiltonian Cycle was correctly defined as a cycle that:

- Visits every vertex exactly once.
- Returns to the starting vertex.

The response correctly identified two conditions before adding a vertex:

1. The vertex must be adjacent to the previous vertex.
2. The vertex must not already have been visited.

An additional final condition was discussed:

After all vertices are included, the final vertex must connect back to the starting vertex.

Evaluation

The answer demonstrated a good understanding of Hamiltonian Cycle and backtracking.

Performance

Excellent

15. Overall Performance

The evaluation demonstrated a strong understanding of fundamental **Design and Analysis of Algorithms** concepts.

The student was able to explain algorithmic concepts in their own words and apply them to practical problems.

Major strengths

- Strong understanding of **backtracking**.
 - Good knowledge of **pruning**.
 - Correct understanding of **state-space trees**.
 - Strong understanding of graph problems.
 - Correct complexity analysis.
 - Correct application of the Master Theorem.
 - Good understanding of computational complexity.
 - Ability to explain algorithms using examples.
 - Good understanding of Greedy algorithms and Dynamic Programming.
-

16. Areas for Improvement

Although the overall performance was strong, the following areas can be improved further:

1. Complexity Analysis

For algorithm questions, always mention both:

Time Complexity

and

Space Complexity

when applicable.

2. NP-Completeness

The definitions should be presented using formal terminology as well as simple explanations.

3. Hamiltonian Cycle

Always explicitly mention the final condition:

Last Vertex $\rightarrow$ Starting Vertex

4. Viva Communication

Answers should be concise and structured:

Definition $\rightarrow$ Approach $\rightarrow$ Example $\rightarrow$ Complexity

This makes technical answers easier to understand during Viva Voce.

17. Key Learning Outcomes

After completing the evaluation, the following learning outcomes were demonstrated:

- Understanding of recursive algorithm design.

- Understanding of backtracking and state-space search.
- Ability to identify and apply pruning.
- Ability to solve graph-based problems conceptually.
- Understanding of NP-completeness.
- Ability to analyze recurrence relations.
- Understanding of asymptotic complexity.
- Ability to distinguish Greedy and Dynamic Programming approaches.
- Ability to explain algorithmic solutions verbally.

18. Viva Preparation Summary

The following points should be remembered for the final Viva:

Topic	Key Point
Backtracking	Build solution step-by-step and backtrack on failure
Pruning	Reject invalid branches early
State-Space Tree	Represents possible solution states
N-Queens	Check column and diagonals
Subset Sum	Include/exclude each element
Graph Coloring	Adjacent vertices need different colors
P	Polynomial-time solvable
NP	Solution can be verified in polynomial time
NP-Hard	At least as hard as NP problems
NP-Complete	Both NP and NP-Hard
Reduction	Transform one problem into another in polynomial time
Master Theorem	Used for divide-and-conquer recurrences
Greedy	Makes locally optimal choices
Dynamic Programming	Uses overlapping subproblems and stored results
Hamiltonian Cycle	Visit every vertex once and return to start

19. Final Conclusion

The DAA mock interview successfully evaluated the student's understanding of important algorithmic concepts.

The student demonstrated strong knowledge of **backtracking, pruning, graph algorithms, computational complexity, recurrence analysis, Greedy algorithms, and Dynamic Programming.**

The recurrence problem was solved correctly using the Master Theorem:

$$T(n) = 2T(n/2) + n$$

resulting in:

$$\boxed{\Theta(n \log n)}$$

Overall, the evaluation indicates that the student has a **good conceptual foundation in Design and Analysis of Algorithms** and is capable of applying algorithmic concepts to different problem scenarios.

20. Final Evaluation

Overall Rating: Very Good / Excellent

Suggested Grade: A

Final Remarks

The student demonstrated good confidence and conceptual understanding throughout the DAA mock interview. The answers showed the ability to understand algorithmic techniques, explain constraints, identify appropriate approaches, and perform complexity analysis. Continued practice with formal definitions, complexity calculations, and concise Viva responses will further strengthen performance.